OperatingSystem :

REAL TIME CPU SCHEDULING

REAL TIME CPU SCHEDULING

❑ Introduction to Real-Time Scheduling

- Delays → system failure / risk
- Focus: strict timing guarantees
- Not just completion, but timely completion

REAL TIME CPU SCHEDULING

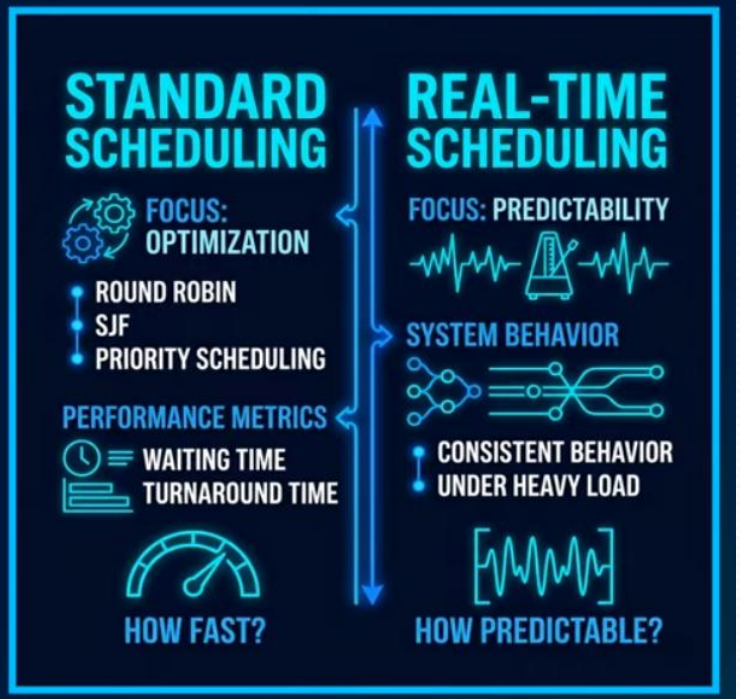
❑ Traffic Signal Analogy

- Precise timing essential
- Small delays → chaos
- Event-triggered tasks
- Immediate response required

REAL TIME CPU SCHEDULING

❑ Comparison with Traditional Scheduling

- Earlier: fairness, efficiency
- Metrics: waiting, turnaround
- Now: predictability over speed
- Consistency under load



REAL TIME CPU SCHEDULING

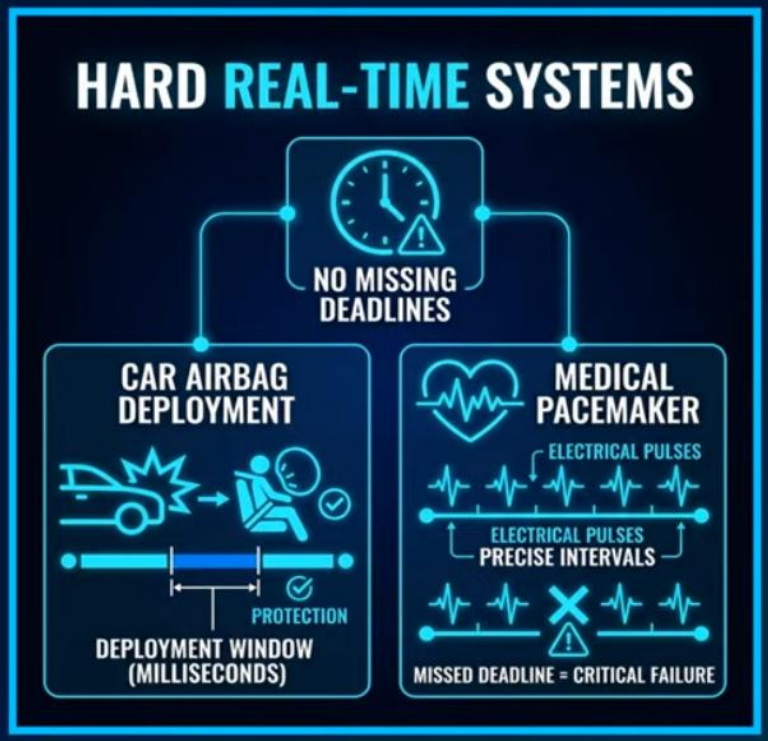
❑ Hard Real-Time System Examples

- Airbag deployment timing
- Pacemakers precision
- No delay tolerance
- Life-critical systems

REAL TIME CPU SCHEDULING

❑ Scheduling in Hard Real-Time Systems

- Deterministic algorithms
- Priority-based execution
- Techniques: RMS, EDF
- Deadlines always guaranteed



REAL TIME CPU SCHEDULING

❑ Soft Real-Time Systems

- Flexible deadlines
- Occasional misses allowed
- Performance degradation only
- Common in daily apps

REAL TIME CPU SCHEDULING

❑ Soft Real-Time Examples

- Video streaming lag
- Online gaming delay
- System still functional
- Quality may drop

REAL TIME CPU SCHEDULING

❑ Hard vs Soft Real-Time Difference

- Hard → strict deadlines
- Soft → flexible targets
- Tolerance in soft systems

REAL TIME CPU SCHEDULING

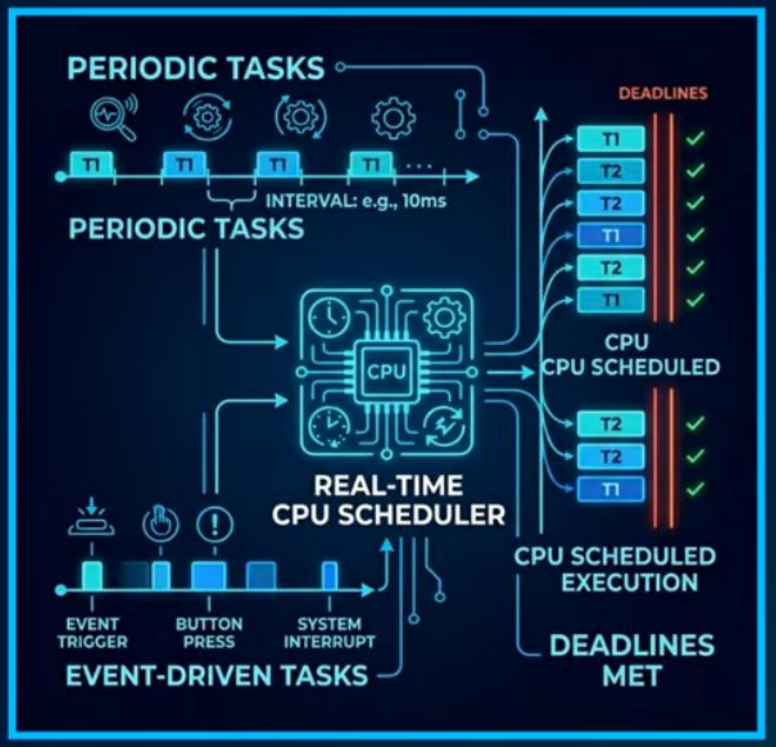
❑ Exam Analogy

- Strict submission → hard RT
- Grace period → soft RT
- Deadline flexibility contrast

REAL TIME CPU SCHEDULING

❑ Types of Tasks in Real-Time Systems

- Periodic tasks → fixed intervals
- Event-driven → triggered actions
- Deadline adherence required



REAL TIME CPU SCHEDULING

❑ Preemptive Scheduling Role

- Interrupt lower-priority tasks
- High-priority executes first
- Critical timing decisions

REAL TIME CPU SCHEDULING

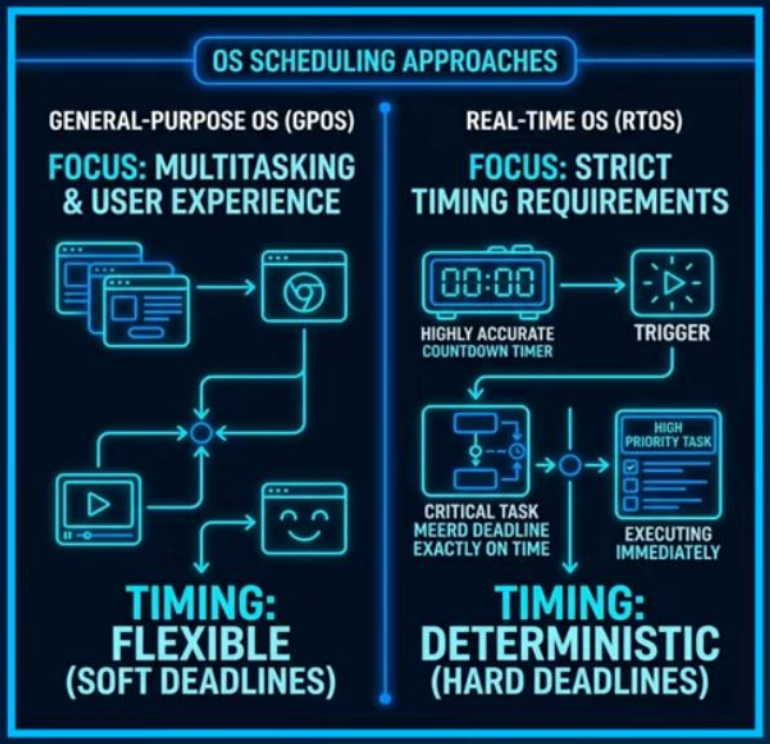
❑ Importance of Latency

- Delay between event & response
- Must be minimal
- Optimize OS for quick reaction

REAL TIME CPU SCHEDULING

❑ Real-Time OS vs General OS

- General OS → multitasking focus
- Real-time OS → timing focus
- Strict execution guarantees



Real Time Systems

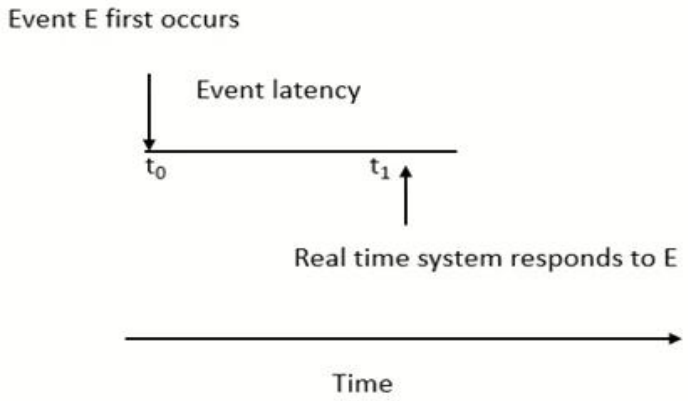
- Event-driven nature
- System typically waits for an event in real time to occur
- Events may arise in:
  - Software: ex. when a timer expires
  - Hardware: ex. a remote-controlled vehicle detects it is approaching obstruction
- When event occurs, system must respond and service it as quickly as possible
- For real-time scheduling, scheduler must support preemptive, priority-based scheduling

Real Time Systems

- **Soft real-time systems**
  - Critical real-time tasks have highest priority
  - No guarantee as to when tasks will be scheduled
  - Guarantee only that process will be given preference over noncritical processes
- **Hard real-time systems**
  - Task must be serviced by its deadline
  - Service after deadline has expired is same as no service

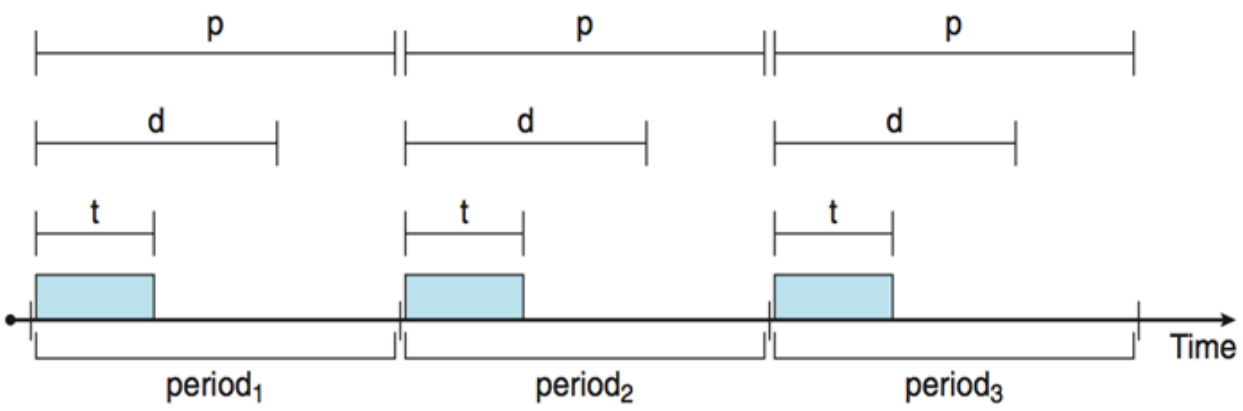
Real Time Systems

- Event latency – amount of time elapse from when an event occurs to when it is serviced
- Different events have different latency requirements



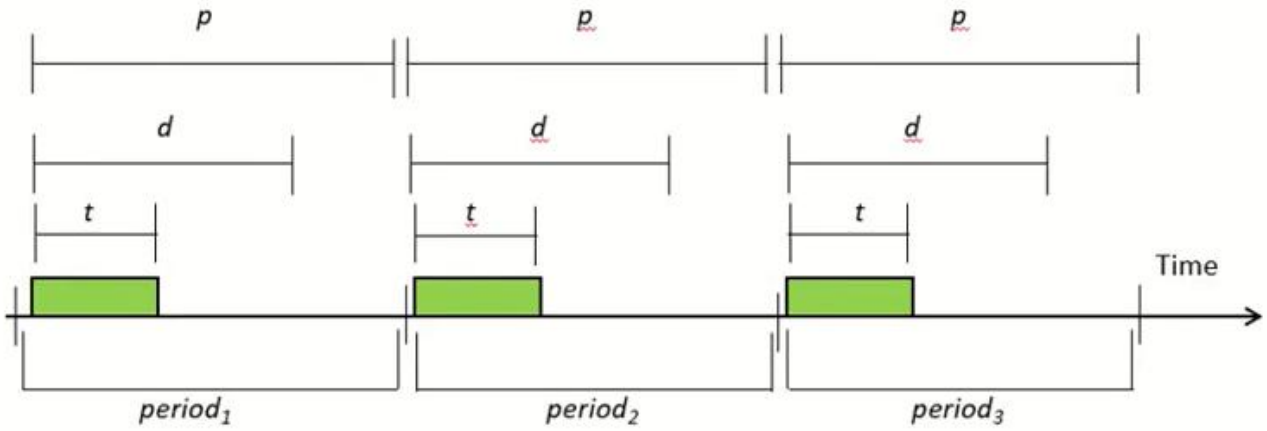
Priority-based Scheduling

- For real-time scheduling, scheduler must support preemptive, priority-based scheduling
  - But only guarantees soft real-time
- For hard real-time must also provide ability to meet deadlines
- Processes have new characteristics: **periodic** ones require CPU at constant intervals
  - Has processing time t, deadline d, period p
  - $0 \leq t \leq d \leq p$
  - **Rate** of periodic task is $1/p$



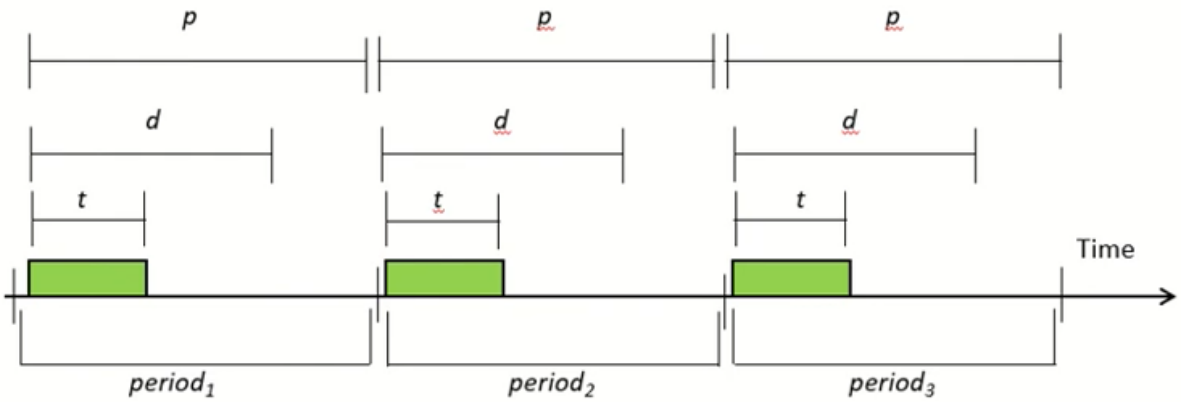
Real Time Systems

- Real-time processes are considered **periodic**
  - Require CPU at constant intervals
  - Has processing time *t*, deadline *d*, period *p*
  - $0 \leq t \leq d \leq p$
  - **Rate** of periodic task is $1/p$



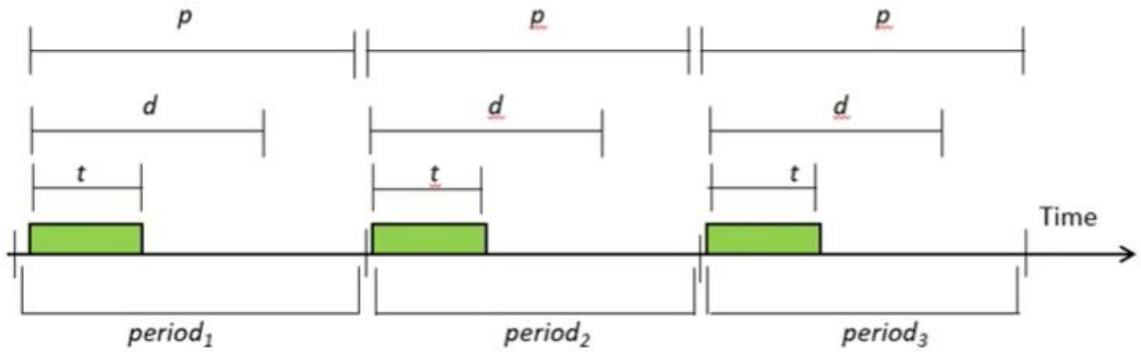
Real Time Systems

- Process may have to announce deadline requirements to scheduler
- Using *admission-control algorithm*, scheduler does one of two things:
  - Admits process - guaranteeing that process will complete on time
  - Rejects request - if cannot guarantee that task will be serviced by its deadline



Real-time Process Scheduling

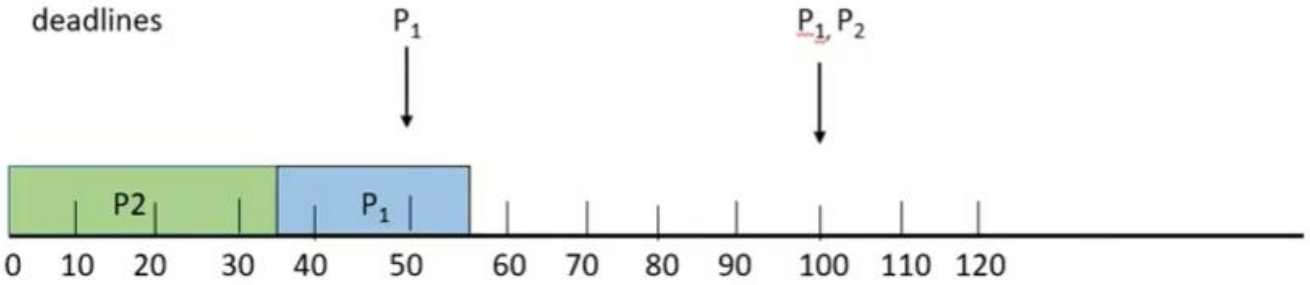
- Schedules periodic tasks using static priority policy with preemption
- Priority assigned based on inverse of its period (Rate = 1/*p*)
  - Shorter periods = higher priority
  - Longer periods = lower priority
  - Rationale - assign higher priority to tasks that require CPU more often
- Assumption - processing time of a periodic process is same for each CPU burst



Missed Deadlines

- Two processes, P1 and P2
  - Periods: p1 = 50, p2 = 100
  - Processing times: t1 = 20, t2 = 35
  - Deadline for each process requires that it complete its CPU burst by start of its next period

If P2 assigned higher priority than P1:



Rate Monotonic Scheduling

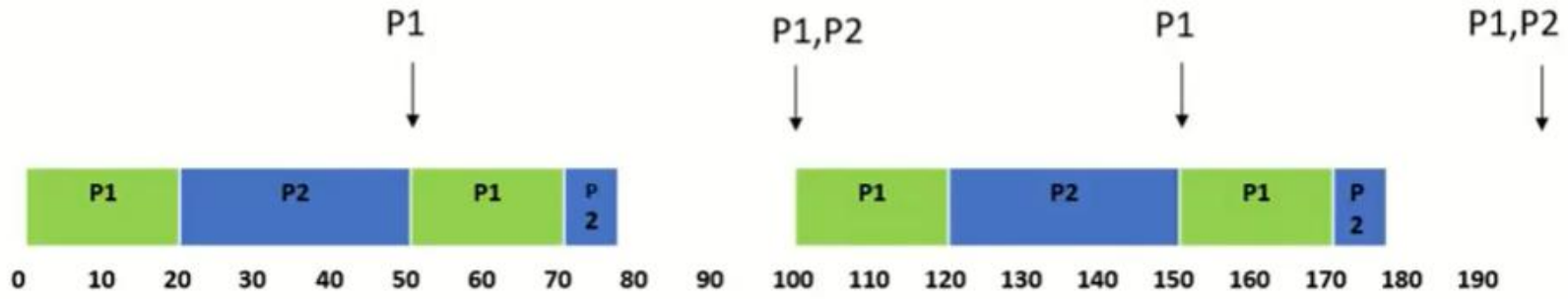
- Is it possible to schedule tasks so that each meets its deadlines?
- Measure CPU utilization of a process P_i as ratio of its burst to its period—t_i / p_i
  - P1: 20/50 = 0.40
  - P2: 35/100 = 0.35
  - Total CPU utilization: 75%
  - Can schedule tasks in such a way that both meet their deadlines and still leave CPU with available cycles

Rate Monotonic Scheduling

P1: p1 = 50, t1 = 20

P2: p2 = 100, t2 = 35

- Period of P1 is shorter than that of P2
- P_1 is assigned higher priority than P_2



Rate Monotonic Scheduling

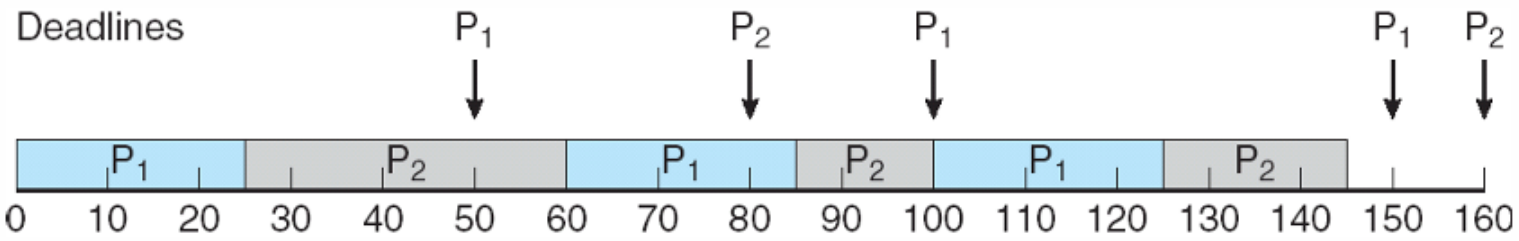
- Rate-monotonic scheduling considered optimal
  - If a set of processes cannot be scheduled by this algorithm, it cannot be scheduled by any other algorithm that assigns static priorities
  - P1: p1 = 50, t1 = 25
  - P2: p2 = 80, t2 = 35
  - Total CPU utilization: (25/50) + (35/80) = 0.94

Earliest Deadline First Scheduling (EDF)

- Priorities are assigned according to deadlines:
  - Earlier the deadline, higher the priority
  - p1 = 50, t1 = 25, p2 = 80, t2 = 35
  - Preemption is not done if deadline is later

the earlier the deadline, the higher the priority;

the later the deadline, the lower the priority



Earliest Deadline First Scheduling (EDF)

- Does not require that processes be periodic, nor must a process require a constant amount of CPU time per burst
- Only requirement - process announce its deadline to scheduler when it becomes runnable